Sockets, HTTP and REST API

Introduction to computer programming

Management and Artificial Intelligence

LUISS

Network Communication

So far...

- we know how to read and write files on our local disk
- we know how to process data stored in our computer
- we know how to make our programs persistent

However, in the modern world, most applications need to communicate over the network:

- web browsers fetch pages from servers
- mobile apps retrieve data from APIs
- IoT devices send sensor data to the cloud

Today we'll learn how Python programs can communicate over the Internet!

Today let's answer the following questions:

1. What are sockets and how do they enable network communication?
2. How can we make HTTP requests to fetch data from the web?
3. What is JSON and why is it so popular for data exchange?
4. How can we handle errors when working with APIs?
5. How do we authenticate with APIs?

Focus on:

1. Basic socket programming concepts
2. The `requests` library for HTTP
3. JSON serialization and deserialization
4. Practical examples with real-world APIs

Sockets: The Foundation

A **socket** is an endpoint for sending or receiving data across a network.

Think of it as a "phone connection" between two computers:

- One computer (server) listens for incoming connections
- Another computer (client) initiates the connection
- Once connected, they can exchange data

Python provides the `socket` module for low-level network programming.

> **NOTE**: Sockets are the foundation of all network communication. HTTP, email, file transfer - they all use sockets under the hood!

Simple Socket Example

```
import socket

# Create a socket object
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to a web server on port 80 (HTTP)
mysock.connect(('data.pr4e.org', 80))

# Send an HTTP request
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)
```

```
47
```

```
while True:
    data = mysock.recv(512) # Receive data
    if len(data) < 1:
        break
    print(data.decode(), end='')

mysock.close()
```

```
HTTP/1.1 200 OK
Date: Mon, 03 Nov 2025 18:47:15 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Sat, 13 May 2017 11:22:22 GMT
ETag: "a7-54f6609245537"
Accept-Ranges: bytes
Content-Length: 167
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: text/plain

But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

NOTE: In the code above:

- `AF_INET` means we're using IPv4 addressing
- `SOCK_STREAM` means we're using TCP (reliable, ordered communication)
- Port 80 is the default port for HTTP
- The `.encode()` method converts strings to bytes (sockets work with bytes)
- The `.decode()` method converts bytes to strings
- The server replies with `HTTP/1.1` even though we asked with `HTTP/1.0`: servers answer with their own protocol version

The Requests Library: Simplifying HTTP

Making HTTP requests with raw sockets is complex. We have to manually format the HTTP request, handle headers, decode responses, etc.

Luckily, Python has libraries that make this much easier!

The most popular is the `requests` library, which provides a simple, intuitive interface for making HTTP requests.

Installing `requests`

The `requests` library is not part of Python's standard library. You need to install it:

```
pip install requests
```

```
Requirement already satisfied: requests in c:\users\ector\miniconda3\lib\si
te-packages (2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\ector\m
iniconda3\lib\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\ector\miniconda3\li
b\site-packages (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\ector\minicon
da3\lib\site-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\ector\minicon
da3\lib\site-packages (from requests) (2025.10.5)
Note: you may need to restart the kernel to use updated packages.
```

Or if you're using conda:

```
conda install requests
```

Making HTTP GET Request

The most common HTTP operation is GET - fetching data from a server. The same functionality as our socket example, but much simpler:

```
import requests

# Make a GET request
response = requests.get('http://data.pr4e.org/romeo.txt')

# The response object contains:
print(response.status_code)  # HTTP status code (200 = success)
print(response.text[:50])    # Response body as text
print(response.headers)      # Response headers
```

```
200
But soft what light through yonder window breaks
I
{'Date': 'Mon, 03 Nov 2025 18:51:22 GMT', 'Server': 'Apache/2.4.52 (Ubunt
u)', 'Last-Modified': 'Sat, 13 May 2017 11:22:22 GMT', 'ETag': '"a7-54f6609
245537"', 'Accept-Ranges': 'bytes', 'Content-Length': '167', 'Cache-Contro
l': 'max-age=0, no-cache, no-store, must-revalidate', 'Pragma': 'no-cache',
'Expires': 'Wed, 11 Jan 1984 05:00:00 GMT', 'Keep-Alive': 'timeout=5, max=1
00', 'Connection': 'Keep-Alive', 'Content-Type': 'text/plain'}
```

HINT: Common HTTP status codes:

- **200**: OK - request succeeded
- **404**: Not Found - resource doesn't exist
- **500**: Server Error - something went wrong on the server
- **401**: Unauthorized - authentication required

JSON: The Universal Data Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format that has become the standard for exchanging data on the web.

Why is JSON so popular?

- It's human-readable
- It's language-independent
- It can represent complex data structures
- It's easy for machines to parse and generate

JSON Syntax

JSON data types map to Python data types:

JSON	Python
`string`	`str`
`number`	`int` or `float`
`boolean`	`bool`
`null`	`None`
`array`	`list`
`object`	`dict`

Example of JSON data:

```
{
    "name": "Alice",
    "age": 30,
    "is_student": false,
    "courses": ["Math", "Physics", "CS"],
    "address": {
        "street": "123 Main St",
        "city": "Rome"
    }
}
```

Working with JSON in Python

Python's `json` module (built-in!) provides functions to convert between JSON strings and Python objects:

```python
import json

# Python dictionary
data = {
    "name": "Alice",
    "age": 30,
    "courses": ["Math", "Physics"]
}

# Convert Python dict to JSON string
json_string = json.dumps(data)
print(json_string)

# Convert JSON string back to Python dict
data_again = json.loads(json_string)
print(data_again)
```

```
{"name": "Alice", "age": 30, "courses": ["Math", "Physics"]}
{'name': 'Alice', 'age': 30, 'courses': ['Math', 'Physics']}
```

Reading and Writing JSON Files

```python
import json

# Write JSON to file
data = {"name": "Bob", "age": 25}
with open("person.json", "w") as f:
    json.dump(data, f)

# Read JSON from file
with open("person.json", "r") as f:
    loaded_data = json.load(f)
    print(loaded_data)  # {'name': 'Bob', 'age': 25}
```

NOTE: Remember the key functions:

- `json.dumps()`: Python object → JSON string (dumps = "dump string")
- `json.loads()`: JSON string → Python object (loads = "load string")
- `json.dump()`: Python object → JSON file
- `json.load()`: JSON file → Python object

REST APIs: Interacting with Web Services

REST (Representational State Transfer) is an architectural style for designing web services.

Most modern web APIs follow REST principles:

- Resources are identified by URLs
- HTTP methods (GET, POST, PUT, DELETE) specify the operation
- Data is exchanged in JSON format
- Each request is independent (stateless)

Fetching Data from a Public API

Let's start with a simple public API that provides interesting data:

```python
import requests

# Fetch a random joke from a public API
response = requests.get('https://official-joke-api.appspot.com/random_joke')

# Check if successful
if response.status_code == 200:
    joke = response.json()  # Automatically parses JSON!
    print(f"Setup: {joke['setup']}")
    print(f"Punchline: {joke['punchline']}")
else:
    print(f"Error: {response.status_code}")
```

```
Setup: What do you call a bee that can't make up its mind?
Punchline: A maybe.
```

HINT: The `requests` library makes it incredibly easy to work with JSON APIs! The `.json()` method automatically converts the JSON response into a Python dictionary.

GET Requests with Parameters

Parameters (also called query parameters or query strings) are used when you need to customize or filter the data you're requesting from an API. Think of them as "options" or "settings" for your request.

```python
import requests

def get_weather(city, units='metric'):
    """Get weather data for a city using Open-Meteo API"""

    # Step 1: Get coordinates for the city
    geocoding_url = 'https://geocoding-api.open-meteo.com/v1/search'
    geocoding_params = {
        'name': city,
        'count': 1
    }

    geo_response = requests.get(geocoding_url, params=geocoding_params, timeout=10)
    geo_response.raise_for_status()
    geo_data = geo_response.json()

    # Check if city was found
    if 'results' not in geo_data or len(geo_data['results']) == 0:
        return None

    # Extract coordinates
    latitude = geo_data['results'][0]['latitude']
```

```
    longitude = geo_data['results'][0]['longitude']
    city_name = geo_data['results'][0]['name']

    # Step 2: Get weather data
    weather_url = 'https://api.open-meteo.com/v1/forecast'
    temp_unit = 'celsius' if units == 'metric' else 'fahrenheit'

    weather_params = {
        'latitude': latitude,
        'longitude': longitude,
        'current_weather': 'true',
        'temperature_unit': temp_unit
    }

    weather_response = requests.get(weather_url, params=weather_params, timeout=10)
    weather_response.raise_for_status()
    weather_data = weather_response.json()

    # Extract temperature
    temperature = weather_data['current_weather']['temperature']

    return {
        'city': city_name,
        'temperature': temperature,
        'units': units
    }

# Test the function
weather = get_weather('Rome')
if weather:
    symbol = '°C' if weather['units'] == 'metric' else '°F'
```

```
    print(f"City: {weather['city']}")
    print(f"Temperature: {weather['temperature']}{symbol}")
    print(f"Units: {weather['units']}")
```

When are parameters used in GET requests?

Common use cases for parameters:

1. **Filtering data** - Narrow down results (e.g., searching for a specific city: `name=Rome`)
2. **Customizing the response** - Control how data is returned (e.g., choosing temperature units: `temperature_unit=celsius`)
3. **Specifying what to retrieve** - Tell the API exactly what data you want (e.g., `current_weather=true`)
4. **Providing identifiers** - Pass coordinates, IDs, or other identifiers (e.g., `latitude=41.9`, `longitude=12.5`)
5. **Pagination** - Request specific chunks of data (e.g., `page=2`, `limit=10`)
6. **Authentication** - Include API keys or tokens (e.g., `api_key=YOUR_KEY`)

Without parameters, you'd get a generic or default response. With parameters, you can ask for exactly what you need!

POST Requests: Sending Data

While GET requests fetch data, POST requests send data to servers.

Creating Data with POST

JSONBin.io requires a free account: copy your **X-Master-Key** from the *API Keys* page and store it in the `JSONBIN_KEY` environment variable.

```python
import os
import requests

def create_bin(data):
    """Create a JSON storage bin using JSONBin.io API"""
    url = 'https://api.jsonbin.io/v3/b'

    # Set headers (JSONBin requires an API key to create a bin)
    headers = {
        'Content-Type': 'application/json',
        'X-Master-Key': os.environ['JSONBIN_KEY']
    }

    try:
        # Make POST request with JSON data
        response = requests.post(url, json=data, headers=headers, timeout=10)
```

```
        # Check if successful
        if response.status_code in [200, 201]:
            # Extract bin ID from the response
            bin_id = response.json()['metadata']['id']
            return {
                'success': True,
                'bin_id': bin_id,
                'url': f'https://api.jsonbin.io/v3/b/{bin_id}',
                'error': None
            }
        else:
            return {
                'success': False,
                'bin_id': None,
                'url': None,
                'error': f'{response.status_code}: {response.text}'
            }

    except requests.exceptions.RequestException as e:
        return {'success': False, 'bin_id': None, 'url': None, 'error': str(e)}

# Test the function
test_data = {
    'name': 'John Doe',
    'age': 30,
    'city': 'Rome'
}

result = create_bin(test_data)
print(f"Success: {result['success']}")
if result['success']:
```

```
    print(f"Bin ID: {result['bin_id']}")
    print(f"URL: {result['url']}")
else:
    print(f"Error: {result['error']}")
```

NOTE: When using `requests.post()` with the `json` parameter:

- The data is automatically serialized to JSON
- The Content-Type header is automatically set to application/json
- You don't need to call `json.dumps()` manually
- Any other header you pass (like `X-Master-Key`) is kept and sent along

Error Handling

Robust applications must handle various types of errors when working with APIs.

Comprehensive Error Handling

```
import requests

def get_weather_safe(city, api_key):
    """Fetch weather data with comprehensive error handling"""
    url = 'https://api.openweathermap.org/data/2.5/weather'

    params = {
        'q': city,
        'appid': api_key,
        'units': 'metric'
    }

    try:
        # Make GET request with timeout
        response = requests.get(url, params=params, timeout=10)

        # Get status code
        status_code = response.status_code

        # Handle different status codes
        if status_code == 200:
            # Success - parse data
```

```
            try:
                data = response.json()
                return {
                    'success': True,
                    'error': None,
                    'status_code': status_code,
                    'data': {
                        'city': data['name'],
                        'temperature': data['main']['temp'],
                        'description': data['weather'][0]['description']
                    }
                }
            except (KeyError, ValueError):
                return {
                    'success': False,
                    'error': 'Failed to parse API response',
                    'status_code': status_code,
                    'data': None
                }

        elif status_code == 401:
            return {
                'success': False,
                'error': 'Invalid API key. Please check your credentials.',
                'status_code': status_code,
                'data': None
            }

        elif status_code == 404:
            return {
                'success': False,
```

```
                'error': f'City "{city}" not found. Please check the spelling.',
                'status_code': status_code,
                'data': None
            }

        elif status_code == 429:
            return {
                'success': False,
                'error': 'Too many requests. Please wait and try again later.',
                'status_code': status_code,
                'data': None
            }

        elif status_code >= 500:
            return {
                'success': False,
                'error': 'Server error. Please try again later.',
                'status_code': status_code,
                'data': None
            }

        else:
            return {
                'success': False,
                'error': f'Unexpected error (status code: {status_code})',
                'status_code': status_code,
                'data': None
            }

    except requests.exceptions.Timeout:
        return {
```

```
            'success': False,
            'error': 'Request timeout. Please check your internet connection.',
            'status_code': None,
            'data': None
        }

    except requests.exceptions.ConnectionError:
        return {
            'success': False,
            'error': 'Connection error. Please check your internet connection.',
            'status_code': None,
            'data': None
        }

    except Exception as e:
        return {
            'success': False,
            'error': f'Unexpected error: {str(e)}',
            'status_code': None,
            'data': None
        }

# Test with invalid API key (401 error)
result = get_weather_safe('London', 'invalid_api_key_12345')
print(f"Success: {result['success']}")
print(f"Status Code: {result['status_code']}")
print(f"Error: {result['error']}")
```

BEST PRACTICES for error handling:

- Always check status codes before processing data
- Provide user-friendly error messages
- Handle network errors (timeout, connection failures)
- Use try-except blocks for JSON parsing
- Set timeouts for requests to prevent hanging

API Authentication

Many APIs require authentication to prevent abuse and track usage.

API Key Authentication

```python
import os
import requests

def get_weather_with_api_key(city, api_key):
    """Get weather data using OpenWeatherMap API with authentication"""
    url = 'https://api.openweathermap.org/data/2.5/weather'

    # Set up parameters including API key
    params = {
        'q': city,
        'appid': api_key,  # API key authentication
        'units': 'metric'
    }

    try:
        # Make GET request with API key in parameters
        response = requests.get(url, params=params, timeout=10)

        # Check if request was successful
        if response.status_code != 200:
            return None
```

```
        # Parse JSON response
        data = response.json()

        # Extract weather information
        return {
            'city': data['name'],
            'temperature': data['main']['temp'],
            'description': data['weather'][0]['description'],
            'humidity': data['main']['humidity']
        }

    except requests.exceptions.RequestException as e:
        print(f"Request failed: {e}")
        return None

API_KEY = os.environ['OPENWEATHER_API_KEY']  # never write the key itself here

weather = get_weather_with_api_key('Rome', API_KEY)
if weather:
    print(f"City: {weather['city']}")
    print(f"Temperature: {weather['temperature']}°C")
    print(f"Description: {weather['description']}")
    print(f"Humidity: {weather['humidity']}%")
```

SECURITY NOTE: Never hardcode API keys in your code! Anyone who sees your code (or your slides, or your GitHub repository) can use your key.

Store the key in an **environment variable** and read it with `os.environ`:

```
# Option A: set it once in the terminal, before starting Jupyter
#   Windows:     setx OPENWEATHER_API_KEY "your-key"   (then reopen the terminal)
#   macOS/Linux: export OPENWEATHER_API_KEY="your-key"

# Option B: type it when the notebook runs (it is never saved in the file)
import os
from getpass import getpass
os.environ['OPENWEATHER_API_KEY'] = getpass('API key: ')
```

NOTE: a new OpenWeatherMap key can take some time to activate: until then the API answers `401`.

SDKs: Even Higher-Level Abstractions

Many popular web services provide **SDKs** (Software Development Kits) that wrap their APIs.

Why use SDKs instead of direct HTTP requests?

- **Simpler code**: No need to construct URLs, headers, or handle raw JSON
- **Type safety**: Many SDKs provide type hints and autocomplete
- **Better error handling**: SDKs often provide meaningful exceptions
- **Built-in features**: Automatic retries, rate limiting, pagination
- **Easier to use**: You work with objects and methods instead of HTTP verbs

Think of it as another level of abstraction:

1. **Sockets** (lowest level): You handle everything manually
2. **requests library**: Simplifies HTTP operations
3. **SDKs** (highest level): Service-specific, easy-to-use interfaces

Example: OpenAI SDK

The OpenAI SDK is a great example of how SDKs simplify working with APIs. Instead of manually constructing HTTP requests to the OpenAI API, you can use the SDK's intuitive interface.

Many other AI providers accept the same requests, so the same SDK also works with them: you only change the server address (`base_url`) and the model name.

Installation:

```
pip install openai
```

Get an API token

To use the OpenAI API, you need an API token. Instead of OpenAI (which is paid), we will use **Groq**, a provider that offers free access to open models and understands the same requests as OpenAI:

1. Sign up for free at console.groq.com (no credit card needed)
2. Open **API Keys** and click **Create API Key**
3. Copy the key (it starts with `gsk_`) and save it in the `GROQ_API_KEY` environment variable

```python
import os
from openai import OpenAI

# Initialize the client with your API key (read from an environment variable)
client = OpenAI(api_key=os.environ["GROQ_API_KEY"],
                base_url="https://api.groq.com/openai/v1")

# Make a chat completion request
prompt = "List the best 5 books for a python developer"
response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": prompt}],
    stream=False,
)
```

```python
# Print the response
print(response.choices[0].message.content)
```

```
Certainly! Here are five highly regarded books for Python developers that c
over various aspects of the language, its ecosystem, and best practices:

1. **"Fluent Python: Clear, Concise Programming" by Luciano Ramalho**
   - This book deep dives into the nuances of Python, focusing on its most
powerful features. It covers data structures, functions, concurrency, and m
etaprogramming, making it a must-read for developers looking to master Pyth
on.

2. **"Python Crash Course: A Hands-On, Project-Based Introduction to Progra
mming" by Eric Matthes**
   - Ideal for beginners, this book presents a practical introduction to Py
thon and programming concepts. It includes projects to build a solid founda
tion and encourages hands-on practice.

3. **"Effective Python: 90 Specific Ways to Write Better Python" by Brett S
latkin**
   - This book provides actionable guidelines and best practices to improve
your Python coding. It covers topics from appropriate use of Python's built
-in features to advanced techniques that can enhance performance and readab
ility.

4. **"Automate the Boring Stuff with Python: Practical Programming for Tota
l Beginners" by Al Sweigart**
   - A fantastic resource for those new to programming, this book teaches h
ow to use Python for everyday tasks, such as web scraping, working with spr
eadsheets, and automating repetitive tasks.
```

5. **"Learning Python" by Mark Lutz**
   - A comprehensive introduction to Python, this book covers fundamentals and advanced topics in a detailed manner. It's great for those who prefer a n in-depth exploration of the language and want a solid reference.

These books cater to a range of skill levels and areas of interest within P ython development, making them essential reads for any developer looking to enhance their skills.

Understanding Each Component

Let's break down what each part does and **why** it's needed:

1. Creating the Client

```
client = OpenAI(api_key="API_KEY_HERE", base_url="https://api.groq.com/openai/v1")
```

- Creates an object that knows how to talk to the OpenAI API
- `api_key`: Your authentication key (like a password)
- `base_url`: The server address where requests are sent
- You create this once and reuse it for multiple requests

2. Making a Request

```
response = client.chat.completions.create(...)
```

This structure tells the API what you want:

- `chat`: Use the chat feature
- `completions`: Get a response/completion
- `create()`: Make a new request

3. Specifying the Model

```
model="openai/gpt-oss-120b"
```

- Different models have different capabilities and speeds
- `openai/gpt-oss-120b` is a free open model from OpenAI, good for most tasks

4. The Messages Format

```
messages=[{"role": "user", "content": "Your question here"}]
```

- **List of dictionaries**: The API needs the conversation history
- `role`: Who's speaking (`"user"`, `"assistant"`, or `"system"`)
- `content`: The actual message text

Example multi-turn conversation:

```
messages=[
    {"role": "system", "content": "You are a helpful Python tutor"},
    {"role": "user", "content": "What's a list?"},
    {"role": "assistant", "content": "A list is a collection..."},
    {"role": "user", "content": "Show me an example"}
]
```

5. Stream Parameter

```
stream=False
```

- `False`: Wait for the complete answer (simpler)
- `True`: Get the response word-by-word as it's typed

6. Getting the Response

```
response.choices[0].message.content
```

The response contains lots of information. We navigate to the part we want: - `choices[0]`: First answer (usually there's only one) - `message.content`: The actual text response

Comparing with Raw Requests

To appreciate why the SDK is useful, let's see how you'd do the **same thing** using the `requests` library directly:

```python
import os
import requests

prompt = "List the best 5 books for a python developer"

# You have to manually construct the URL
url = "https://api.groq.com/openai/v1/chat/completions"

# You have to manually create the headers with authentication
headers = {
    "Authorization": f"Bearer {os.environ['GROQ_API_KEY']}",
    "Content-Type": "application/json"
}

# You have to manually structure the request body as a dictionary
data = {
    "model": "openai/gpt-oss-120b",
    "messages": [{"role": "user", "content": prompt}],
    "stream": False
}
```

```python
# Make the POST request
response = requests.post(url, headers=headers, json=data)

# Check if the request was successful
if response.status_code == 200:
    # You have to manually parse the JSON response
    result = response.json()
    # You have to know the exact structure of the response
    print(result["choices"][0]["message"]["content"])
else:
    print(f"Error: {response.status_code}")
    print(response.text)
```

What the SDK Does for You

Notice the differences? The SDK handles all these details automatically:

Task	Raw Requests	SDK
URL Construction	Manual string building	Handled automatically
Authentication	Create headers dictionary	Pass `api_key` parameter
Request Formatting	Build JSON dictionary manually	Use method parameters
Error Handling	Check status codes manually	Raises meaningful exceptions
Response Parsing	Navigate raw JSON dictionary	Access response object properties
Type Hints	No autocomplete	IDE suggests valid parameters

Lines of code: ~30 lines with requests vs. ~5 lines with SDK!

BEST PRACTICE: When a service provides an official SDK, use it! The SDK:

- Saves you time and reduces errors
- Handles API changes automatically (just update the SDK)
- Provides better error messages
- Includes documentation and examples
- Often includes helpful features like automatic retries

Common HTTP Status Codes

Understanding HTTP status codes is essential for debugging API issues:

Success Codes (2xx)

- **200 OK**: Request succeeded
- **201 Created**: Resource successfully created
- **204 No Content**: Success, but no content to return

Client Error Codes (4xx)

- **400 Bad Request**: Invalid request syntax
- **401 Unauthorized**: Authentication required or failed
- **403 Forbidden**: Server understood request but refuses
- **404 Not Found**: Resource doesn't exist
- **429 Too Many Requests**: Rate limit exceeded

Server Error Codes (5xx)

- **500 Internal Server Error**: Generic server error
- **502 Bad Gateway**: Invalid response from upstream server
- **503 Service Unavailable**: Server temporarily unavailable

Additional Resources

- Requests Documentation
- HTTP Status Codes
- REST API Tutorial
- JSON Documentation
- Public APIs List
- OpenAI API Documentation
- Groq OpenAI compatibility